

Proving the Lottery Ticket Hypothesis: Pruning is All You Need

Eran Malach, Gilad Yehudai, Shai Shalev-Shwartz, Ohad
Shamir

Talk by Gleb Karpeev

Motivation: Why Prune Neural Networks?

Standard Practice

- ▶ Train large over-parameterized networks
- ▶ Prune up to 90% of weights with minimal accuracy loss
- ▶ Inference becomes much cheaper

Surprising Observation (Frankle & Carbin, 2018)

Training a pruned network from scratch fails.

But pruned subnetworks can be retrained successfully *if initialized to their original random values*.

Core Question

Does a random network *already contain* a good subnetwork inside?

The Lottery Ticket Hypothesis

Original Hypothesis (Frankle & Carbin, 2018)

A randomly-initialized network contains a subnetwork that - when trained in isolation - matches the accuracy of the full network.

Strong Hypothesis (Ramanujan et al., 2019)

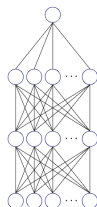
A sufficiently over-parameterized random network contains a subnetwork that achieves competitive accuracy without any training.

This Paper's Contribution

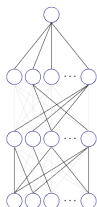
Theoretical proof of the strong hypothesis:

For any target network with bounded weights, a polynomially-larger random network contains a good subnetwork with high probability.

Two Types of Subnetworks



(a) Full Network



(b) Weight Subnetwork



(c) Neuron Subnetwork

Figure: (a) Full random network; (b) Weight-subnetwork; (c) Neuron-subnetwork.

Weight-subnetwork:

- ▶ Mask individual weights
- ▶ More expressive

Neuron-subnetwork:

- ▶ Remove entire neurons
- ▶ Hardware-friendly

Main Result: Weight Pruning (Theorem 2.1)

Theorem (informal)

Let F be any target ReLU network of depth l with bounded weights.

Let G be a random network of depth $2l$ and width $\text{poly}(d, n, l, 1/\epsilon, \log(1/\delta))$, with weights $\sim U([-1, 1])$.

Then w.p. $\geq 1 - \delta$ there exists a weight-subnetwork $\tilde{G}$ of G such that:

$$\sup_{x \in \mathcal{X}} \left| \tilde{G}(x) - F(x) \right| \leq \epsilon$$

Key Takeaway

A $2\times$ deeper random network is enough to approximate any target!
Number of active weights $O(dn + n^2l)$ - same order as in F .

Proof Intuition: The Key Identity

ReLU Decomposition

For any scalar a , using $\sigma(x) = \max\{x, 0\}$:

$$a = \sigma(a) - \sigma(-a)$$

Consequence

Every ReLU neuron $\sigma(\langle w^*, x \rangle)$ has a depth-2 representation:

$$\sigma\left(\sum_{i=1}^d \sigma(w_i^* x_i) - \sum_{i=1}^d \sigma(-w_i^* x_i)\right)$$

Construction

For each coordinate i , find *two* random hidden neurons whose weights are close to $\pm w_i^*$. Pruning everything else yields the approximation.

By concentration: width $\gg d/\epsilon$ suffices.

Corollaries of Weight Pruning

1. Universal Approximation via Pruning

Combined with classical universal approximation:

For every smooth f , a large enough random network contains a subnetwork that approximates f to arbitrary accuracy.

2. Pruning is Computationally Hard

Reduction from hardness of training depth-2 ReLU networks:

- ▶ Optimal pruning $\geq$ optimal training
- ▶ Training is NP-hard $\Rightarrow$ optimal pruning is NP-hard

Practical Implication

Existence is guaranteed, but practical heuristics are needed.

Analogous to gradient descent for training: heuristic, not optimal.

Neuron Pruning $\equiv$ Random Features (Theorem 3.2)

Random Features Model

Sample $w_1, \dots, w_k \sim \mathcal{D}^*$, train only the second layer:

$$f(x) = \sum_{i=1}^k u_i \sigma(\langle w_i, x \rangle)$$

Equivalence

Pruning entire neurons is equivalent to random features:

A neuron-subnetwork achieves small loss $\iff$ some random features model does.

Consequence

Neuron pruning inherits the *limitations* of random features:

Cannot efficiently approximate a single ReLU neuron (Yehudai & Shamir, 2019).

When Neuron Pruning Works (Theorem 3.4)

Despite its limitations, neuron pruning suffices in two important settings:

1. Overfitting a Finite Sample

For dataset $S = \{(x_i, y_i)\}_{i=1}^m$ with kernel matrix of minimal eigenvalue $\lambda > 0$:

Width $k = \text{poly}(m, 1/\lambda, 1/\epsilon)$ guarantees a neuron-subnetwork that fits S .

2. Learning RKHS Functions

For f in the RKHS of the associated kernel:

Width $k = \text{poly}(C, 1/\epsilon)$ guarantees a neuron-subnetwork approximating f .

Notable

Width depends on *target complexity*, not on input dimension d .

Weight vs. Neuron Pruning

Aspect	Weight-pruning	Neuron-pruning
Expressive power	Any ReLU network	Random features only
Depth required	$2l$ for depth- l target	Shallow only
Single ReLU neuron	Efficiently approx.	Requires exp. width
Input dim. dependence	Polynomial in d	Independent of d
Equivalent to	Weight optimization	Linear/kernel models

Key Insight

Weight-pruning is *strictly stronger* than neuron-pruning.
Neuron-pruning cannot escape the random features regime.

Conclusion

Main Contribution

First theoretical proof of the strong lottery ticket hypothesis:
Any target network can be approximated by pruning a random network of polynomially larger size.

Key Takeaways:

- ▶ Pruning is as powerful as training (in terms of expressivity)
- ▶ Architecture matters: weight-pruning $>$ neuron-pruning
- ▶ Optimal pruning is computationally hard - same status as training
- ▶ Motivates developing pruning-based algorithms

Future Directions:

- ▶ Heuristic algorithms with provable guarantees
- ▶ Improving polynomial dependencies on d, n, l
- ▶ Extension to convolutional layers and ResNets